

Multi-Organ CT Segmentation with SAM 2 and LoRA



Summary

This project adapts SAM 2 to segment abdominal organs in 3D CT scans. The key idea: a CT volume's axial slices are treated as the frames of a video, so SAM 2's memory, built to track an object through time, tracks an organ through depth.

The image encoder was fine-tuned with LoRA on all 13 BTCV organs, training only 119,808 parameters (0.26% of the 46.2M-parameter SAM 2.1 Hiera-Small model). On the first organ evaluated, the liver, Dice on 6 held-out scans rose from 0.733 to 0.963 and HD95 fell from 230.9 mm to 5.9 mm.

Each 3D organ mask comes from a single box prompt on one axial slice, which SAM 2's video predictor propagates slice by slice in both directions through the volume. The prompts are derived from ground-truth labels, so these are oracle-prompt results (see Limitations).

Liver, 6 val cases	Zero-shot SAM 2	SAM 2 + LoRA
Dice (mean $\pm$ SD)	0.733 $\pm$ 0.144	0.963 $\pm$ 0.003
HD95, mm (mean $\pm$ SD)	230.9 $\pm$ 45.6	5.9 $\pm$ 1.8

Background: SAM 2

[SAM 2](#) (Ravi et al., Meta, 2024) is a promptable segmentation model for images and videos. Given a box or click on an object, it returns that object's mask, and in a video it keeps tracking the object from frame to frame. It was trained on SA-V, which Meta describes as the largest video segmentation dataset to date.

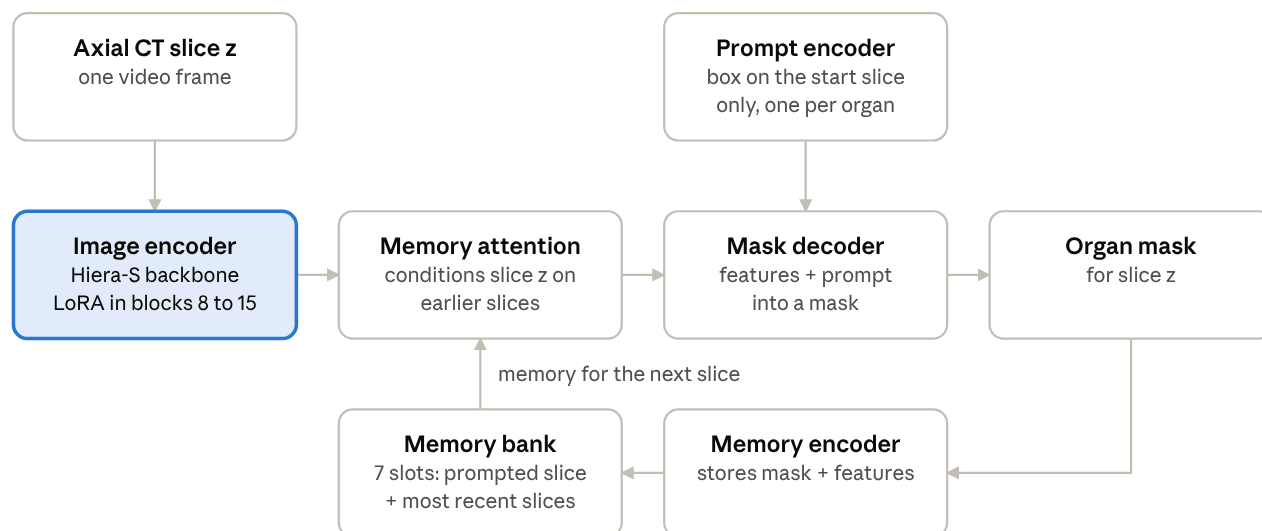
Its parts, and the role each plays here:

- **Image encoder (Hiera):** turns each frame into multi-scale features, once per slice. The LoRA adapters live here.

- **Prompt encoder:** embeds the box drawn on the organ's start slice.
- **Mask decoder:** combines image features and the prompt into a mask.
- **Memory encoder and memory bank:** each predicted mask is encoded and stored. The bank keeps the prompted frame plus the most recent frames, 7 slots in the SAM 2.1 config.
- **Memory attention:** conditions the current frame on the bank, so a mask carries from slice to slice with no new prompts.

Each slice is segmented using memory of the slices already segmented

SAM 2 applied to CT: axial slices play the role of video frames



SAM 2 components as used here · LoRA sits in the image encoder

Why video mode fits CT. Neighbouring axial slices change gradually, much like consecutive video frames, so an organ outlined once can be followed through the depth of the volume. One prompt per organ per scan replaces one prompt per slice, and the memory keeps the 3D mask consistent along z.

Data and preprocessing

The data is BTCV (the MICCAI 2015 "Beyond the Cranial Vault" abdominal CT set), labelled for 13 organs: spleen, right and left kidney, gallbladder, esophagus, liver, stomach, aorta, inferior vena cava, portal and splenic vein, pancreas, and right and left adrenal gland. Scans `img0035` to `img0040` are held out for evaluation; every other scan in the training folder is used for training.

Volume preprocessing (shared by training and evaluation)

1. Load each NIfTI volume with nibabel and reorient it to the closest canonical (RAS) orientation, so the third axis is always the axial (z) direction.
2. Clip intensities to a soft-tissue window of -150 to 250 HU and rescale to $0-255$ (uint8).
3. Keep the voxel spacing from the header; it is needed later to report HD95 in millimetres.

Training slices

1. Keep axial slices that contain at least one organ, taking every 2nd one (`SLICE_STRIDE = 2`), since neighbouring slices are nearly identical.
2. Store each slice once, at its original resolution, together with its full 13-organ label map.
3. At training time, on the GPU: upsample to 1024×1024 with bicubic interpolation (the same resampling SAM 2's own frame loader uses), copy the grey channel to RGB, and apply ImageNet mean/std normalization, which SAM 2's pretrained encoder expects.

Evaluation frames. Each validation volume is written as one JPEG per axial slice, the format SAM 2's video predictor reads. The predictor resizes and normalizes the frames itself, so training and evaluation see identically prepared images.

Prompting

Each organ gets exactly one prompt per scan: a bounding box on one chosen axial slice, the start slice. Both the slice and the box come from the ground-truth label.

Choosing the start slice. The simplest choice, the slice where the organ's cross-section is largest, can land on a slice where the organ appears as several disconnected pieces. One box around all of them is loose, and SAM 2 locks on poorly. The pipeline instead:

1. takes the 5 slices with the largest organ area;
2. picks the one with the fewest connected components;
3. breaks ties by compactness (organ area $\div$ bounding-box area), then by area.

The box. The tightest box around the organ on that slice, padded by 4 pixels on each side and clipped to the image. Training uses the same rule for its box prompts, padded in original-resolution pixels and then scaled to 1024×1024 , so the model is trained on the kind of prompt it later receives.

LoRA fine-tuning

Only the image encoder is adapted, with low-rank adapters ([LoRA](#), Hu et al., 2021); every original SAM 2 weight stays frozen. Each adapted linear layer computes its frozen output plus a small trainable correction:

$$h = W_0 x + \frac{\alpha}{r} B A x, \quad A \in \mathbb{R}^{r \times d_{in}}, \quad B \in \mathbb{R}^{d_{out} \times r}$$

B starts at zero, so training begins from exactly the pretrained model. Adapters sit in the attention `qkv` projection of Hiera blocks 8 to 15, the later half of the 16-block encoder. The prompt encoder, mask decoder, memory modules and blocks 0 to 7 are untouched. Only 119,808 of 46.2M parameters train (0.26%), and a checkpoint holds just the adapter weights.

Training step

1. Load a batch of 4 CT slices and encode each one once.
2. For every organ present on a slice (up to 6, a random subset each epoch), build its box prompt and decode a mask from that slice's shared encoding.
3. Compare each predicted mask with its ground truth at the decoder's native 256×256 resolution, using binary cross-entropy plus soft Dice. Dice is computed per mask and then averaged, so small organs such as the adrenal glands weigh as much as the liver.

Training is single-frame. It adapts how the encoder sees CT, and the video memory is used unchanged at inference.

Setting	Value
Base model	SAM 2.1 Hiera-Small (<code>sam2.1_hiera_small.pt</code>)
LoRA rank r , scale α	8, 16
Adapted layers	attention <code>qkv</code> , blocks 8 to 15 of 16
Optimizer	AdamW, learning rate $1e-4$, weight decay $1e-4$
Batch	4 slices, up to 6 organ prompts each
Epochs	up to 10, capped by a 3-hour time budget
Precision	bfloat16 where supported, else float16 with loss scaling

Setting	Value
Slice sampling	every 2nd organ-containing slice

Making training feasible. A first version treated every organ on every slice as its own sample. Each slice then passed through the encoder once per organ, and 10 epochs were estimated at about 15 hours. Encoding each slice once for all its organs removes that repeated work, which was the main saving. Using every 2nd slice, adapting only the later encoder blocks (about 15% cheaper per step, measured) and computing the loss at 256×256 add further savings.

Open question: the GPU, number of epochs completed and total training time should be copied in from the training log.

3D inference: propagation along z

A full 3D organ mask comes from SAM 2's video predictor, run over the axial slices as if they were frames in time. For each scan and organ:

1. Build the video predictor from the base SAM 2.1 checkpoint. For the fine-tuned model, insert LoRA adapters into the same encoder blocks used in training and load the trained weights. Loading fails with an error if any adapter weight does not match.
2. Initialise the predictor's state on the scan's JPEG frames. Frames and state are kept on the CPU to fit GPU memory.
3. Add the box prompt on the start slice.
4. Propagate forward from the start slice to the last slice, then backward from the start slice to the first. Each new slice is conditioned on the memory bank of slices already segmented.
5. Threshold each slice's mask logits at 0, resize to the scan's original in-plane size, and stack the slices into a 3D mask.
6. Reset the predictor's memory before the next scan.

The zero-shot baseline runs the same steps with the same prompts, without adapters. The two models differ only in their encoder weights, so any change in the results comes from fine-tuning.

Evaluation protocol

Every metric is computed on the full 3D mask of each held-out scan, at the scan's original resolution. So far the liver (organ 6) has been evaluated on 6 scans, `img0035` to `img0040`.

Dice similarity coefficient measures volume overlap, from 0 (none) to 1 (perfect), for prediction P and ground truth G :

$$\mathrm{DSC} = \frac{2|P \cap G|}{|P| + |G|}$$

HD95 measures boundary error in millimetres. For every surface voxel of each mask, take the distance to the nearest surface voxel of the other mask. HD95 is the 95th percentile of all those distances, using the scan's voxel spacing. It ignores the worst 5% of boundary points, but unlike Dice it heavily penalises fragments far from the organ.

Checks on the result

- **No train/validation overlap.** None of the 6 validation scans appear in the training folder (filename check). The script also refuses to train on validation cases and compares file contents to catch renamed copies.
- **No empty predictions.** Every case produced a non-empty mask for both models, so all 6 cases count in every mean. An empty mask would have an infinite HD95 and be reported separately.
- **Same prompts for both models.** Zero-shot and fine-tuned runs use the identical start slice and box.

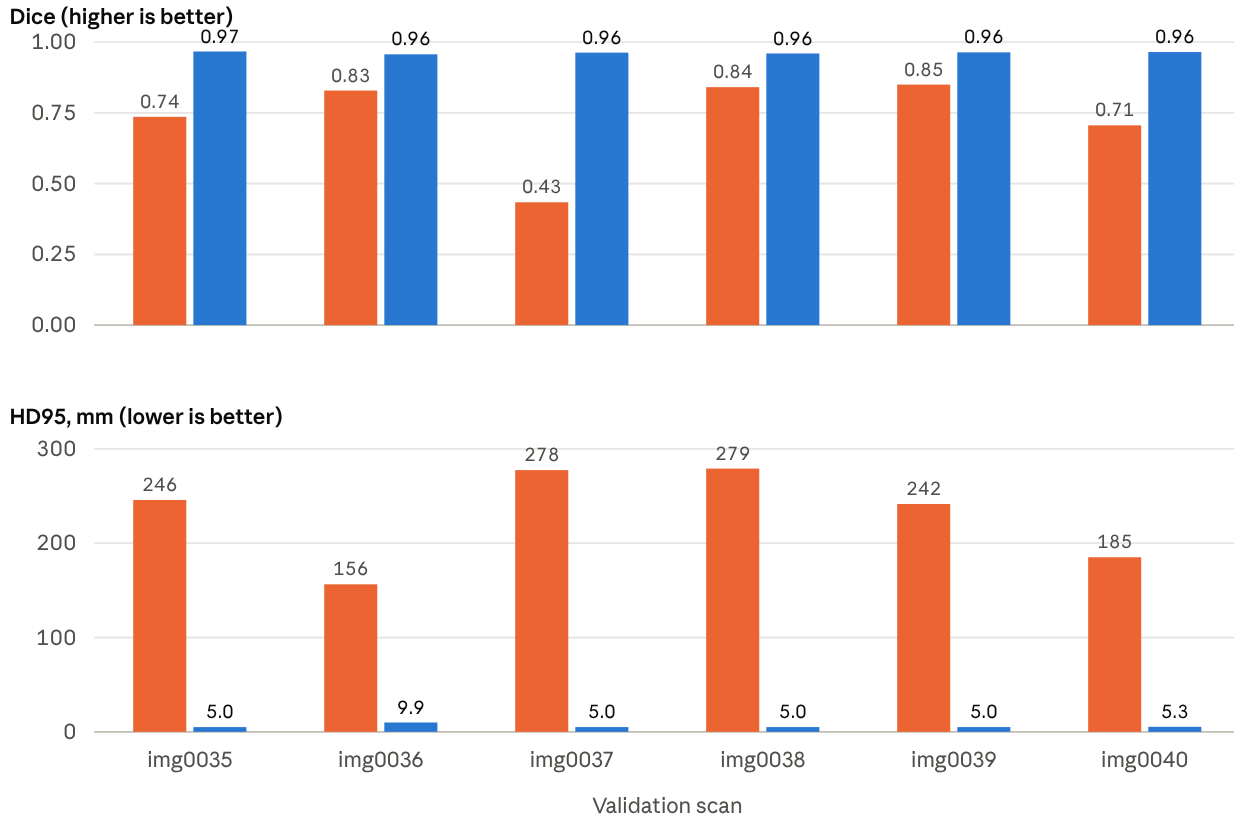
Results: liver

Fine-tuning improved every validation scan, and made the model far more consistent: LoRA Dice spans only 0.957 to 0.967, while zero-shot spans 0.434 to 0.850. The means are in the summary table at the top.

Fine-tuning lifted every case above 0.95 Dice and below 10 mm HD95

Liver, 6 held-out BTCV scans

Zero-shot SAM 2 SAM 2 + LoRA



eval_organ6.csv · liver · 6 validation scans · values rounded

- **The weakest zero-shot case recovered fully.** `img0037` went from 0.434 to 0.963 Dice.
- **Zero-shot's HD95 comes from stray fragments.** `img0038` has a reasonable zero-shot Dice of 0.841 but an HD95 of 279 mm. The liver outline is mostly right, but propagation also marked small regions far from the liver, likely on slices above or below it. Dice barely registers small distant fragments; HD95 does. Fine-tuning removed them, bringing HD95 to 5.0 to 9.9 mm.
- **Four LoRA cases have HD95 of exactly 5.0 mm.** This most likely equals those scans' slice spacing, meaning the remaining error is about one slice at the organ's upper or lower edge. The spacing is printed in the evaluation log and should be confirmed there.

Limitations

The results are strong, but they come from a narrow evaluation and should be read as a promising first result, not a general claim.

- **Oracle prompts.** The start slice and box both come from the ground-truth label, so these scores are an upper bound. In practice a clinician or a detector would draw the box, less precisely. How much the results drop with imperfect boxes has not been tested.
- **Only 6 validation scans.** The spread is tight, but 6 cases give a wide confidence interval, and there is no cross-validation or separate untouched test set.
- **Only one organ evaluated.** The model was trained on all 13 organs but has only been tested on the liver, the largest and highest-contrast one. Small or thin structures such as the pancreas, adrenal glands, gallbladder, esophagus and veins are where SAM 2 usually struggles most, and are untested.
- **The video memory was not trained.** Fine-tuning adapts only the image encoder, on single slices. Memory attention and the memory encoder still run with their original video-trained weights, so propagation errors may remain for organs that change shape quickly from slice to slice.
- **One box per organ per scan.** An organ that appears as several separate pieces on the start slice gets a single box around all of them, which may segment them poorly.
- **No tuning or ablations.** This is a single training run with fixed settings: LoRA rank, adapted blocks, slice stride and number of epochs. Their effect on accuracy has not been measured.
- **No supervised baseline on the same split.** The comparison is with zero-shot SAM 2 only. Comparisons with published results for models like nnU-Net are not like-for-like: those use different splits and no prompts.

Next steps

1. **Evaluate all 13 organs** with the existing checkpoint; no retraining is needed. Set `ORGANS_TO_EVAL = list(range(1, 14))` in the config.
2. **Test with imperfect prompts** by shifting and resizing the ground-truth boxes, to measure how the results hold up without an oracle.
3. **Enlarge the evaluation** with more held-out scans or cross-validation.
4. **Train the memory path** on short runs of consecutive slices, so propagation itself adapts to CT.
5. **Run ablations** over LoRA rank, the number of adapted blocks and slice stride.

Reproducing the results

The whole pipeline is one script, `sam2_btcv_colab.py`, also split into a `src/` package with `scripts/train.py` and `scripts/evaluate.py`. On a Colab GPU runtime it mounts Google Drive, installs SAM 2, downloads the Hiera-Small checkpoint, trains, and evaluates. Paths, validation cases, organs and all settings live in one config block.

Training resumes from the last saved epoch after a disconnect. Random seeds are not fixed, so a rerun will give slightly different numbers.

Sources

- Ravi et al., [SAM 2: Segment Anything in Images and Videos](#), 2024
- Hu et al., [LoRA: Low-Rank Adaptation of Large Language Models](#), 2021
- Meta, [SAM 2 code and checkpoints](#)